

Experimental Report

Submodular Optimization and Greedy Approximation for Distributed Influence Maximization

Yixin Chen Bichi Zhang Yaoqin Ye

CS240: Algorithm Design and Analysis, Spring 2026

Abstract

This project studies influence maximization, a classical network optimization problem that asks how to choose a small number of seed nodes so that a diffusion process reaches as many nodes as possible. Following the CS240 project requirements, we connect a modern data application to classical algorithms: monotone submodular maximization, greedy approximation, lazy evaluation, and a communication-constrained distributed variant. We implemented Independent Cascade simulation, random/degree/PageRank baselines, centralized Monte Carlo greedy, CELF lazy greedy, and a GreeDI-style local candidate sharing method. Experiments were run on synthetic and small real-world benchmark graphs. The results show that structural baselines are surprisingly strong on small graphs, that CELF reduces the number of influence evaluations while preserving the same algorithmic objective, and that distributed candidate sharing exposes a clear tradeoff among solution quality, runtime, and communication budget.

1 Background and Related Work

Influence maximization is a standard problem in social networks, viral marketing, public-health intervention, and misinformation analysis. Given a network and a limited budget, the goal is to choose seed nodes that maximize the expected number of eventually activated nodes. The problem is algorithmic rather than purely empirical: the chosen seed set must solve a combinatorial optimization problem under uncertainty.

Kempe, Kleinberg, and Tardos [1] introduced influence maximization under the Independent Cascade (IC) and Linear Threshold (LT) models. They showed that the problem is NP-hard, but that the expected spread function is monotone and submodular. Nemhauser, Wolsey, and Fisher’s classical result [2] then implies that greedy maximization under a cardinality constraint obtains a $(1 - 1/e)$ approximation for the exact submodular objective.

The difficulty is that exact influence spread is expensive to compute. A straightforward implementation estimates the spread of many candidate seed sets using Monte Carlo simulation. Leskovec et al. [3] proposed CELF, a lazy greedy method that uses submodularity to avoid many repeated marginal-gain evaluations. For large-scale or distributed settings, Mirzasoleiman et al. [4] proposed GreeDI, which runs greedy locally on partitions and then merges local representatives. More recent methods such as TIM/IMM [5, 6] use reverse influence sampling to scale influence maximization further; in this project, those methods are treated as related work rather than full reproduction targets.

2 Problem Formulation

Let $G = (V, E, p)$ be a directed graph. Each edge (u, v) has an activation probability $p_{uv} \in [0, 1]$. Given a seed budget k , the influence maximization problem is

$$\max_{S \subseteq V, |S| \leq k} \sigma(S),$$

where $\sigma(S)$ is the expected number of active nodes after diffusion starts from seed set S .

Our implementation uses the Independent Cascade model. When a node u first becomes active, it gets one chance to activate each inactive out-neighbor v with probability p_{uv} . Each activation attempt is independent, and the process stops when no newly active nodes remain.

Under the IC model, σ is monotone and submodular:

$$\sigma(A \cup \{v\}) - \sigma(A) \geq \sigma(B \cup \{v\}) - \sigma(B), \quad A \subseteq B, v \notin B.$$

This diminishing-returns property is the reason greedy selection has a theoretical guarantee and the reason CELF can safely treat old marginal gains as upper bounds.

3 Representative Method Analysis

3.1 Centralized Greedy

The centralized greedy algorithm starts with $S_0 = \emptyset$. At round t , it chooses

$$v^* = \arg \max_{v \in V \setminus S_t} (\hat{\sigma}(S_t \cup \{v\}) - \hat{\sigma}(S_t)),$$

where $\hat{\sigma}$ is the Monte Carlo estimate of IC spread. The seed set is updated as $S_{t+1} = S_t \cup \{v^*\}$ until $|S| = k$.

If the graph has n nodes and m edges, one IC simulation costs $O(n + m)$ in the worst case. With R Monte Carlo simulations per estimate, naive greedy costs approximately

$$O(knR(n + m)),$$

ignoring small savings from already selected nodes. This is feasible for small course-project graphs but becomes expensive quickly.

3.2 CELF Lazy Greedy

CELF maintains a priority queue of cached marginal gains. Since submodularity implies that a node's marginal gain cannot increase as the seed set grows, an old marginal gain is an upper bound on its current gain. When the top node in the queue was last evaluated for the current seed-set size, CELF accepts it. Otherwise, it recomputes only that node's marginal gain and pushes it back. The selected seed set targets the same greedy objective but typically requires far fewer spread evaluations.

3.3 GreeDI-Style Distributed Greedy

For the distributed component, the node set is partitioned into m local parts. Worker i runs greedy over its local candidate list and returns at most q candidates. The coordinator forms

$$C = \bigcup_{i=1}^m C_i$$

and runs a final greedy selection over C rather than all nodes. We test $m \in \{2, 4, 8\}$ and $q \in \{k, 2k, 5k\}$. The communication proxy is the number of transmitted candidates, approximately mq but capped by partition sizes.

4 Reproduction and Implementation

The source code is organized as follows:

- `src/simulation.py`: Independent Cascade simulation and Monte Carlo spread estimation.
- `src/baselines.py`: random, weighted out-degree, and PageRank seed selection.
- `src/greedy.py`: centralized greedy and CELF lazy greedy.
- `src/distributed.py`: random node partitioning and GreeDI-style candidate sharing.
- `src/graphs.py`: benchmark graph construction and weighted-cascade probability assignment.
- `src/experiments.py`: end-to-end experiment runner and CSV output.

Because exact reproduction of large influence maximization systems is beyond the scope of the course project, we reproduced the core algorithmic claims at a smaller scale: IC spread estimation, greedy submodular selection, CELF lazy evaluation, and distributed local-candidate merging. The experiment runner used six cases: Erdos–Renyi graphs with 40 and 70 nodes, Barabasi–Albert graphs with 40 and 70 nodes, a Watts–Strogatz graph with 60 nodes, and Zachary’s Karate Club graph with 34 nodes. The seed budget was $k = 5$ or $k = 6$ depending on the case.

Each algorithm’s final seed set was re-evaluated using a common Monte Carlo seed for the same graph case. This makes the reported spread more comparable than the internal spread estimates used during selection.

5 Experimental Evaluation

5.1 Overall Algorithm Comparison

Table 1 summarizes the average results across all graph cases. The reported ratio is relative to centralized greedy on the same graph.

Table 1: Average performance across all benchmark cases.

Algorithm	Spread	Ratio	Runtime (s)	Evaluations
Random	16.58	0.93	0.00	1.0
Degree	24.42	1.37	0.00	1.0
PageRank	22.92	1.27	0.00	1.0
Greedy	18.08	1.00	0.06	283.5
CELf	18.92	1.04	0.02	112.0
Best distributed average setting	24.00	1.34	0.32	359.5

The most important observation is that the small-scale experiment does not show a clean dominance of greedy over structural baselines. Degree and PageRank perform very well on these graphs, especially because the weighted-cascade probability rule favors high-degree structural positions. This is not a contradiction of the greedy approximation theorem: the theorem compares greedy to the optimal seed set for the exact submodular objective, while our implementation uses noisy Monte Carlo estimates during selection and then re-evaluates final seed sets with another random seed.

CELf behaves as expected: it obtains similar average spread to greedy but uses fewer spread evaluations, reducing the average from 283.5 to 112.0 evaluations. This directly demonstrates the benefit of lazy evaluation.

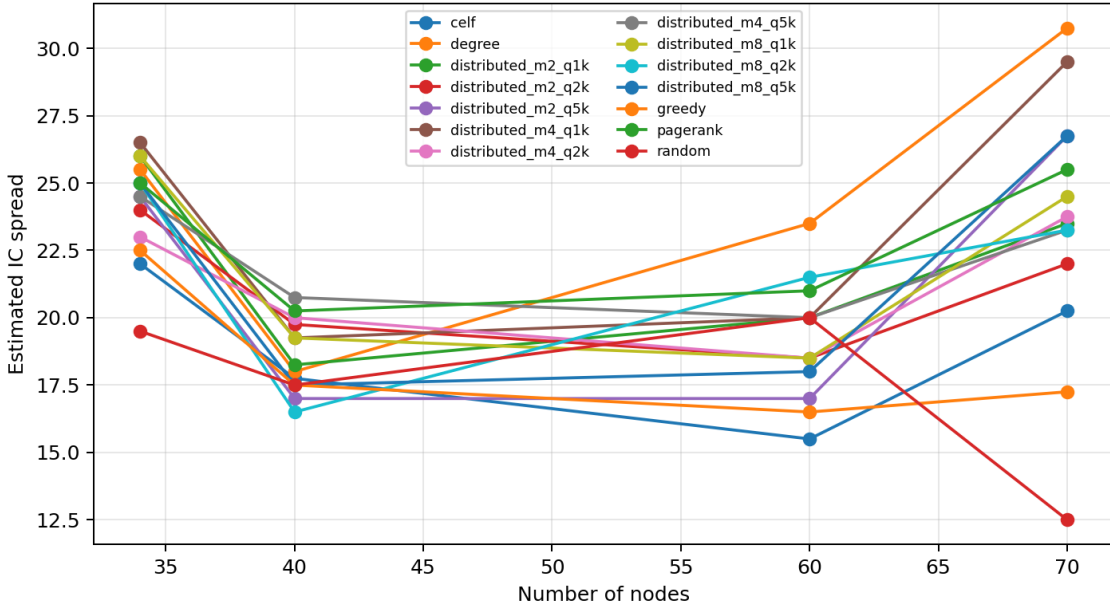


Figure 1: Estimated IC spread by graph size and algorithm.

5.2 Distributed Communication Tradeoff

Table 2 summarizes the distributed variants. The notation $(m, q/k)$ means m partitions and local candidate budget $q = (q/k) \cdot k$.

Table 2: Average distributed performance under different partition and communication budgets.

Setting	Spread	Ratio	Runtime (s)	Evaluations	Candidates
(2, 1)	21.58	1.19	0.30	321.5	11.0
(2, 2)	21.00	1.16	0.64	591.5	22.0
(2, 5)	21.50	1.20	1.37	1021.5	49.0
(4, 1)	24.00	1.34	0.32	359.5	22.0
(4, 2)	21.50	1.18	0.61	599.8	43.0
(4, 5)	22.08	1.22	0.71	683.8	52.3
(8, 1)	22.00	1.21	0.42	430.5	43.0
(8, 2)	21.00	1.16	0.58	503.2	52.3
(8, 5)	21.92	1.22	0.51	503.2	52.3

The distributed results show the expected cost side of the tradeoff clearly: larger local budgets generally increase the number of transmitted candidates and spread evaluations. The quality side is noisier. In several cases, distributed candidate sharing beats the centralized greedy baseline after fair re-evaluation. This can happen because local greedy introduces diversity into the candidate pool and because the centralized greedy baseline is selected under noisy Monte Carlo estimates. The result should therefore be interpreted as evidence that candidate sharing can be competitive on small graphs, not as a claim that it theoretically dominates centralized greedy.

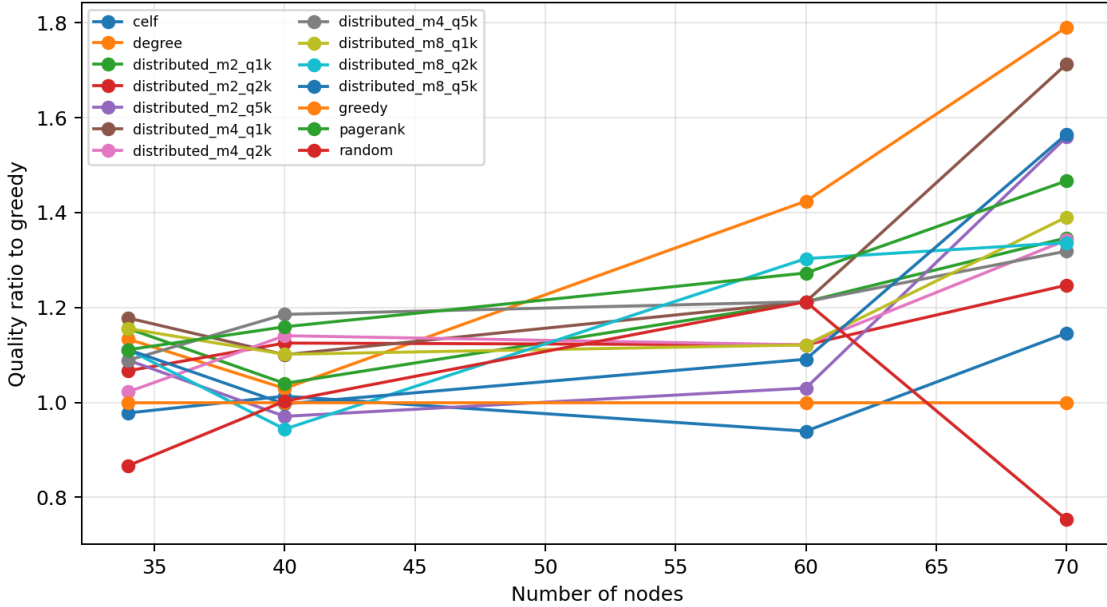


Figure 2: Quality ratio relative to centralized greedy.

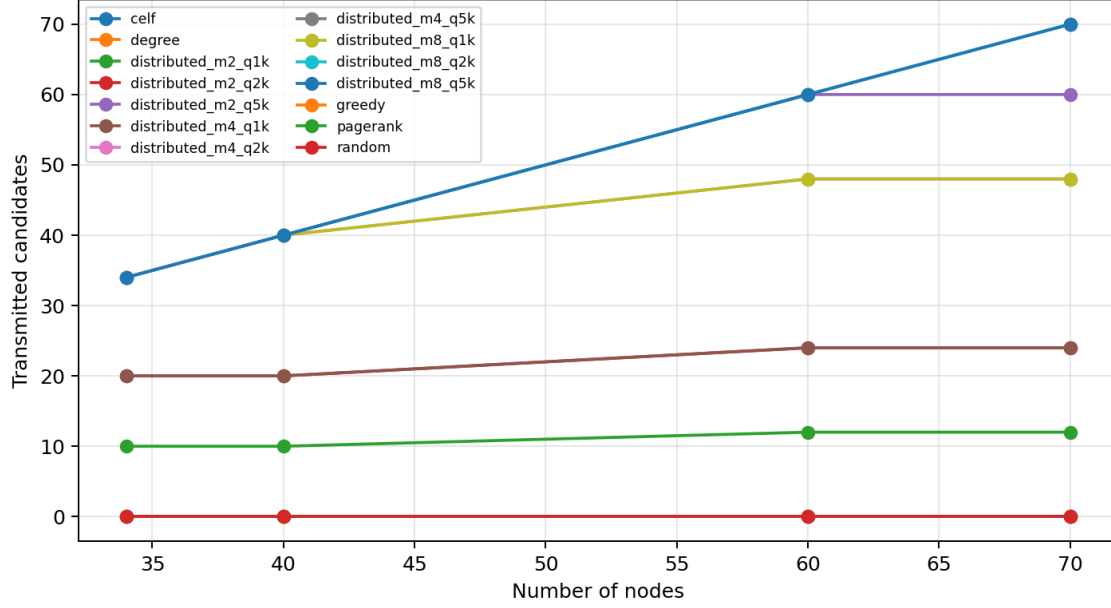


Figure 3: Communication proxy: number of transmitted candidates.

5.3 Runtime and Evaluation Counts

Centralized greedy has a small wall-clock time on these graphs, but its evaluation count grows with k and the number of candidate nodes. CELF reduces the number of evaluations by reusing marginal-gain upper bounds. Distributed variants perform more total evaluations because each worker runs a local greedy phase and the coordinator runs another greedy phase. This is acceptable only when worker computations are parallelized or when full-graph candidate search is impossible due to memory or ownership constraints.

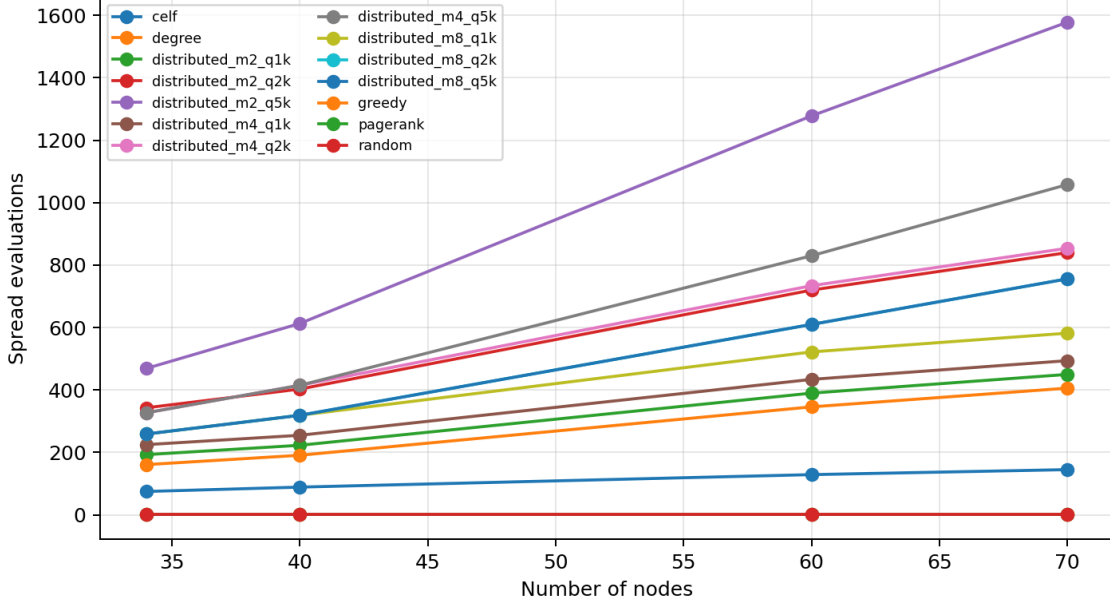


Figure 4: Number of spread evaluations. CELF reduces evaluations compared with naive greedy.

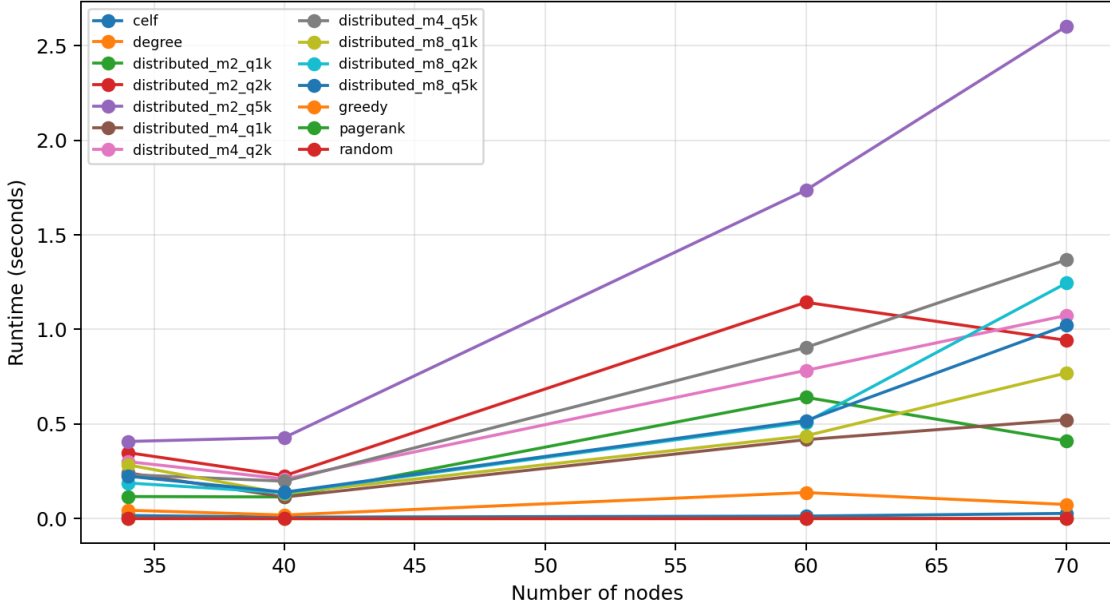


Figure 5: Wall-clock runtime on the current benchmark set.

6 Deviations, Limitations, and Optional Extensions

The proposal mentioned larger synthetic graphs and possible real networks such as Wiki-Vote or NetHEPT. The current reproduction uses smaller graphs so that the whole experiment can run

quickly in the course-project environment. This choice keeps the implementation transparent, but it limits scalability claims.

Another limitation is Monte Carlo noise. The experiment uses a modest number of simulations for speed, and distributed variants use fewer simulations during their local selections. As a result, the selected seed set may be sensitive to randomness. This explains why the fair re-evaluation sometimes ranks degree or distributed variants above centralized greedy. A stronger final experiment would increase the simulation count, average over multiple random seeds, and report confidence intervals.

The implemented optional extension is the GreeDI-style communication-constrained variant. Further possible extensions include deterministic partition strategies, larger real datasets, RR-set based maximum coverage, and parallel worker execution to measure true distributed speedup.

7 Conclusion

This project demonstrates how a modern network problem can be understood using classical algorithmic ideas. Influence maximization under IC diffusion gives a monotone submodular objective; greedy selection provides a theoretical approximation guarantee; CELF turns submodularity into a practical acceleration; and GreeDI-style candidate sharing connects the problem to distributed optimization under communication limits. The empirical results are intentionally small-scale, but they reproduce the central algorithmic mechanisms and expose the main practical issue: influence maximization quality depends not only on the approximation algorithm, but also on spread-estimation accuracy, graph structure, and communication constraints.

References

- [1] D. Kempe, J. Kleinberg, and E. Tardos. Maximizing the spread of influence through a social network. *KDD*, 2003.
- [2] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher. An analysis of approximations for maximizing submodular set functions. *Mathematical Programming*, 1978.
- [3] J. Leskovec, A. Krause, C. Guestrin, C. Faloutsos, J. VanBriesen, and N. Glance. Cost-effective outbreak detection in networks. *KDD*, 2007.
- [4] B. Mirzasoleiman, A. Karbasi, R. Sarkar, and A. Krause. Distributed submodular maximization: Identifying representative elements in massive data. *NeurIPS*, 2013.
- [5] Y. Tang, X. Xiao, and Y. Shi. Influence maximization: Near-optimal time complexity meets practical efficiency. *SIGMOD*, 2014.
- [6] Y. Tang, Y. Shi, and X. Xiao. Influence maximization in near-linear time: A martingale approach. *SIGMOD*, 2015.